

APRENDIZAJE AUTOMÁTICO I

Grado en Ciencia de Datos e Inteligencia Artificial

PRÁCTICA 1

Aprendizaje Supervisado — Clasificación de Salarios

Curso 2025-2026

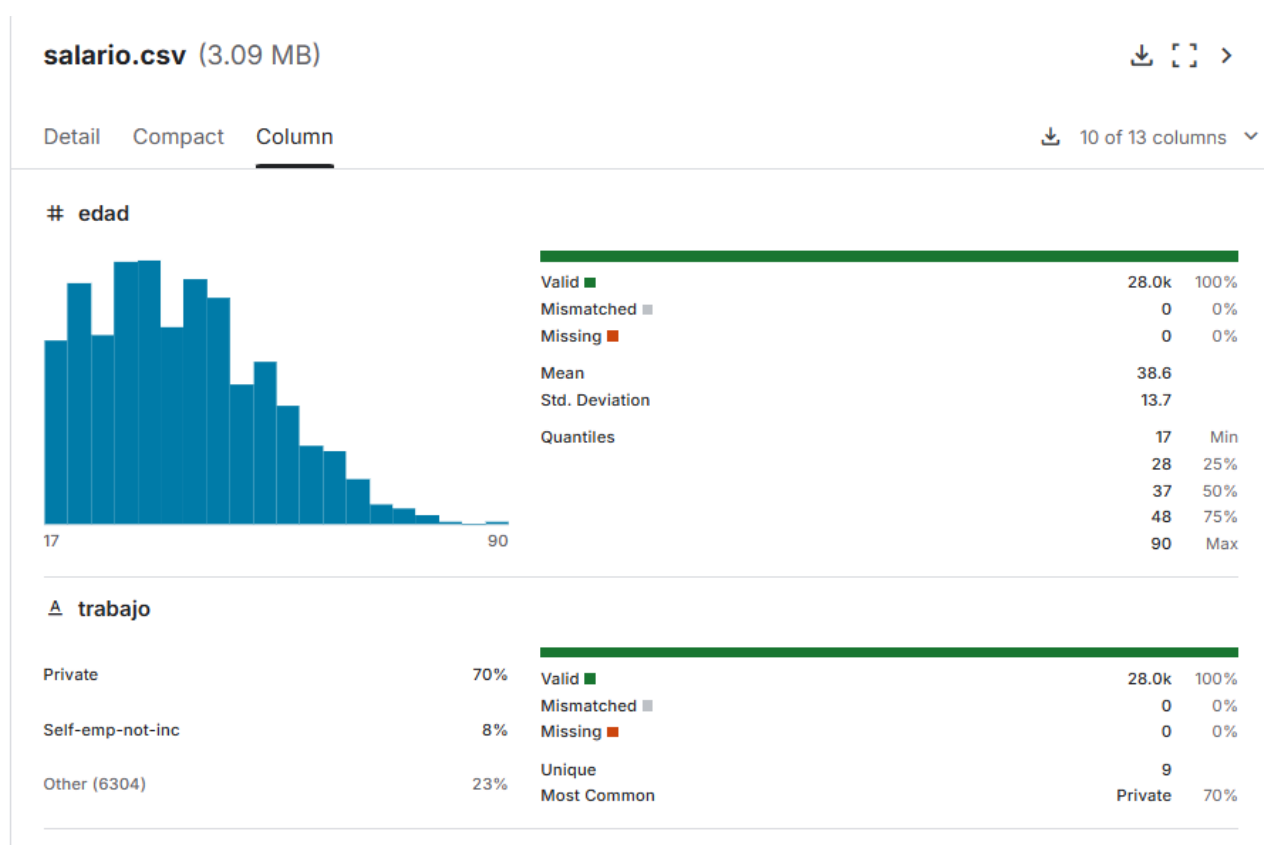
Índice

1. Introducción y descripción del problema
2. Parte I: Preprocesado (preprocess.ipynb)
 - 2.1 Exploración inicial y limpieza de datos
 - 2.2 Transformación y codificación de variables
 - 2.3 Evolución del enfoque de codificación: de one-hot a df_expanded
 - 2.4 Selección de características: método filtro
 - 2.5 División train / validación / test
 - 2.6 Método wrapper: búsqueda greedy bidireccional
 - 2.7 Método embebido: SelectFromModel
 - 2.8 Justificación de la elección del método filtro
3. Parte II: Modelos supervisados (supervised.ipynb)
 - 3.1 Estrategia general: división y tratamiento del desbalanceo
 - 3.2 Justificación del uso de GridSearchCV
 - 3.3 Métodos clásicos: regresiones lineal, polinomial y logística
 - 3.4 Árboles de decisión y modelos ensemble
 - 3.5 Aprendizaje basado en instancias: KNN
 - 3.6 Máquinas de Soporte Vectorial (SVM)
 - 3.7 Redes neuronales (MLP)
 - 3.8 Comparativa global y elección del mejor modelo
4. Parte III: Mejor modelo (best_model.ipynb)
 - 4.1 Justificación de Random Forest como modelo final
 - 4.2 Análisis de importancia de características
 - 4.3 Generación de predicciones para Kaggle
5. Conclusiones

1. Introducción y descripción del problema

El objetivo de esta práctica es abordar un problema clásico de clasificación binaria supervisada: predecir si una persona percibe un salario anual superior o inferior a 50.000 dólares a partir de un conjunto de características sociodemográficas y laborales. El dataset de partida, `salario.csv`, contiene 13 columnas (12 predictores y la etiqueta objetivo), con instancias que describen edad, nivel de estudios, estado civil, ocupación, horas trabajadas semanalmente, país de origen y otros atributos de cada persona.

La práctica se estructura en tres notebooks claramente diferenciados que replican el flujo estándar de un proyecto de Machine Learning: preprocesado exhaustivo de los datos (`preprocess.ipynb`), evaluación comparativa de múltiples modelos supervisados con búsqueda de hiperparámetros (`supervised.ipynb`) y generación de predicciones finales con el mejor modelo encontrado (`best_model.ipynb`). La métrica de evaluación principal es el F1-score weighted average, que pondera el F1 de cada clase por su soporte y resulta más adecuada que el accuracy simple en presencia del desbalanceo de clases presente en el dataset (aproximadamente 75% de instancias con salario $\leq 50K$ frente al 25% con salario $> 50K$).



2. Parte I: Preprocesado (preprocess.ipynb)

El preprocesado es la fase más determinante de cualquier proyecto de aprendizaje automático. El objetivo es transformar los datos brutos en una representación adecuada para que los algoritmos de ML puedan extraer patrones de forma efectiva, eliminando información perjudicial y garantizando la calidad del conjunto de entrenamiento. Este notebook recorre cuatro grandes etapas: limpieza, transformación, selección de características y división de conjuntos.

2.1 Exploración inicial y limpieza de datos

La exploración inicial reveló que el dataset contiene 27.998 instancias con 12 características. Los valores faltantes no aparecen como NaN estándar sino codificados como el carácter '?'. Tras identificarlos y sustituirlos por NaN, se comprobó que únicamente tres columnas presentaban datos faltantes: `origen_trabajo`, `trabajo` y `pais_origen`, con aproximadamente un 6% de valores faltantes en cada una.

La estrategia de imputación se decidió de forma independiente para cada columna, analizando su distribución empírica. Para `origen_trabajo` y `pais_origen`, la categoría más frecuente (Private y United-States, respectivamente) concentraba aproximadamente el 70% de las observaciones, por lo que imputar con la moda no introduce sesgo apreciable sobre la distribución real. Para `trabajo`, ninguna categoría mostraba dominancia clara, razón por la que se optó por asignar una categoría explícita Unknown: esta decisión preserva la fila sin alterar la distribución de las categorías existentes ni introducir ruido.

Finalmente se eliminaron los registros duplicados encontrados, paso fundamental de higiene antes de cualquier transformación. Eliminar duplicados evita que el modelo sobrepondere ciertas instancias y que la evaluación posterior resulte optimista.

2.2 Transformación y codificación de variables

La transformación de variables es la etapa de mayor impacto sobre la calidad de la representación que recibe el modelo.

One-Hot Encoding crea una columna binaria por cada categoría. No introduce orden artificial y permite al modelo tratar cada categoría como una entidad independiente, que es la representación semánticamente correcta para variables nominales. Este fue el enfoque aplicado en el notebook a la mayoría de las variables categóricas.

El criterio aplicado para cada variable fue el siguiente:

- **Edad:** se normalizó dividiendo entre 100 para llevarla al rango [0, 1]. Se eligió esta normalización manual frente a `MinMaxScaler` porque este último es muy sensible a outliers; si el conjunto de test presentara un valor extremo nuevo, el rango cambiaría y la escala sería incompatible. Dividir entre 100 garantiza una escala estable e idéntica en train y test.
- **Horas de trabajo semanales:** se estandarizó mediante `StandardScaler` (media cero, desviación uno). El scaler se ajustó exclusivamente sobre `salario.csv` y se aplicó con `transform` puro sobre `salario_test.csv`, evitando data leakage.
- **Ganancias y pérdidas por inversiones:** ambas variables presentan distribuciones altamente sesgadas con más del 90% de valores en cero y outliers extremos (hasta 99.999 en ganancias). Se aplicó binning mediante `pd.cut`, creando categorías ordinales significativas (sin ganancias, ganancias moderadas, altas) que capturan la estructura real de la variable sin que los outliers distorsionen el espacio de features.
- **Sexo:** mapeo directo a 0/1 por ser variable binaria.

- Estudios: aunque se aplica One-Hot Encoding en la primera representación, la variable tiene un orden natural implícito (Preschool < 1st-4th < ... < Doctorate). El OHE con categorías definidas en orden respeta esta jerarquía al expandirla.
- Variables categóricas nominales (origen_trabajo, estado_civil, trabajo, posicion_familiar, etnia, pais_origen): One-Hot Encoding con el conjunto de categorías fijado explícitamente mediante `pd.Categorical`. Esta fijación es fundamental para garantizar que el espacio de features del conjunto de test sea idéntico al de entrenamiento, independientemente de si alguna categoría no aparece en el test.

2.3 Evolución del enfoque de codificación: de one-hot a df_expanded

Un aspecto relevante del proceso de preprocesado fue la evolución en la representación interna de las variables codificadas. En la implementación inicial, las variables categóricas tras aplicar `pd.get_dummies` quedaban almacenadas como listas de enteros dentro de columnas individuales del DataFrame: cada celda contenía un vector como `[1, 0, 0, 0, 0, 0, 0, 0]` en lugar de columnas separadas para cada categoría. Esta representación compacta resulta cómoda para visualizar el DataFrame y comprobar las transformaciones, pero presenta un problema crítico: los modelos de scikit-learn y la mayoría de las funciones de selección de características esperan un DataFrame con una columna por feature, no columnas que contengan listas.

Al intentar aplicar los métodos de selección de características (`f_classif`, `chi2`, `r_regression` de `sklearn.feature_selection`), se hizo evidente que esta representación no era compatible con las APIs esperadas. La solución fue construir `df_expanded`: un nuevo DataFrame en el que cada elemento de los vectores OHE se expande a una columna separada con nombre descriptivo (por ejemplo, `origen_trabajo_Private`, `estudios_Bachelors`, `estado_civil_Married-civ-spouse`, etc.). Esta expansión se realiza mediante la función `expandir_ohe` definida en el notebook.

El cambio de representación no modifica la información contenida; simplemente la transforma a la forma tabular estándar que requieren los algoritmos. El resultado es un dataset de aproximadamente 103 features (variables continuas/discretas originales más todas las columnas OHE expandidas), que es el punto de partida real para la selección de características y el entrenamiento de modelos.

2.4 Selección de características: método filtro

Existen tres grandes familias de métodos de selección de características con propiedades muy distintas. El método filtro (aproximación indirecta) evalúa la relevancia de cada variable de forma independiente mediante criterios estadísticos, sin intervención de ningún modelo de ML. El método wrapper (aproximación directa) evalúa subconjuntos entrenando un modelo real con cada subconjunto. Los métodos embebidos realizan la selección durante el propio entrenamiento del modelo.

Para el método filtro se aplicaron tres tests estadísticos distintos en función de la naturaleza de cada variable, siguiendo el criterio de coherencia entre el tipo de variable y el test apropiado:

Variable	Tipo	Test estadístico	Criterio de selección
edad, horas_trabajo_semana	Continua	Pearson (<code>r_regression</code>)	$ r > 0.1$
sexo, ganancias, pérdidas	Ordinal/discreta	F-score (<code>f_classif</code>)	p-valor < 0.05
Variables OHE (nominales)	Binaria	Chi-square (<code>chi2</code>)	p-valor < 0.05

El criterio $p\text{-valor} < 0.05$ corresponde al nivel de confianza estadístico estándar: se rechaza la hipótesis nula de independencia entre la variable y la etiqueta con una probabilidad de error del 5%. En la práctica, esto significa que solo se retienen aquellas variables para las que existe evidencia estadística suficiente de que están asociadas con la variable objetivo, eliminando ruido y reduciendo la dimensionalidad de forma objetiva y reproducible.

El resultado fue la selección de 74 features de las aproximadamente 103 disponibles en `df_expanded`. Estas 74 variables constituyen el dataset procesado (`processed_dataset.csv`) que se usa como punto de partida en `supervised.ipynb`.

2.5 División train / validación / test

La división del dataset en conjuntos disjuntos es un requisito fundamental para garantizar que la evaluación de los modelos sea representativa del rendimiento real sobre datos no vistos. Se utilizó una división en tres conjuntos con proporciones 70/15/15, aplicada con `shuffle=True` y `random_state=42` para garantizar la reproducibilidad.

La razón para dividir en tres conjuntos y no solo dos (train/test) está directamente ligada a los requisitos del workflow de entrenamiento. El conjunto de validación cumple un papel diferente al de test: se usa durante el entrenamiento de redes neuronales como señal para el early stopping (detener el entrenamiento cuando el rendimiento en validación deja de mejorar, evitando overfitting), mientras que el test se reserva como estimación imparcial del rendimiento final una vez elegidos los hiperparámetros. Mezclar ambos roles en un único conjunto produciría una evaluación sesgada.

La división se realizó en dos pasos para obtener exactamente las proporciones deseadas: primero se separó el 15% destinado a test, y luego se separó el 15% de validación del 85% restante (lo que equivale aproximadamente al 17.6% del conjunto train+val). La semilla `random_state=42` garantiza que todos los modelos en `supervised.ipynb` se evalúan sobre exactamente el mismo conjunto de test, haciendo la comparativa final válida.

2.6 Método wrapper: búsqueda greedy bidireccional

El método wrapper, aproximación directa donde cada subconjunto candidato de variables se evalúa mediante el modelo clasificatorio inducido a partir de esas variables, fue implementado mediante la variante BG (Bidirectional Generation), que combina dos búsquedas greedy simultáneas para capturar tanto las variables más relevantes individualmente como las que aportan valor en el contexto del conjunto completo.

La búsqueda SFG (Sequential Forward Generation) parte del conjunto vacío y añade iterativamente la feature que más mejora el F1-score macro en validación cruzada de 3 folds. La SBG (Sequential Backward Generation) parte del conjunto completo y elimina la feature menos informativa en cada paso. El subconjunto BG final es la unión de los dos soportes resultantes.

El resultado fue revelador: el modelo entrenado con todas las features obtuvo un rendimiento superior al modelo entrenado con cualquier subconjunto seleccionado por el wrapper. Este resultado, aunque aparentemente sorprendente, es coherente con lo que las diapositivas advierten sobre los métodos wrapper: tienen riesgo de overfitting en la selección y su resultado depende fuertemente del clasificador base elegido. En este caso, con el `RandomForestClassifier` como estimador interno, la búsqueda greedy encontró subconjuntos que funcionan bien con ese modelo concreto pero no necesariamente mejoran la representación general.

2.7 Método embebido: `SelectFromModel`

El método embebido realiza la selección de características durante el propio proceso de entrenamiento, aprovechando información interna del modelo. Una ventaja clave que estos métodos son más rápidos que los wrapper y tienen en cuenta las interacciones entre características.

Se implementó mediante `SelectFromModel` con un `RandomForestClassifier` entrenado sobre `X_train`. El umbral de selección se fijó en la media de las importancias de features (`threshold='mean'`): todas las variables cuya importancia (reducción media de impureza Gini) supera la media se retienen. El resultado fue una selección de aproximadamente 27 features.

Al igual que con el wrapper, los modelos entrenados sobre este subconjunto no superaron al entrenado con las 74 features del filtro. El método embebido produce una selección específicamente optimizada para el Random Forest que genera las importancias, lo que puede no ser óptimo para otros clasificadores como las redes neuronales o las SVM.

2.8 Justificación de la elección del método filtro

Tras evaluar los tres métodos, se eligió el filtro como base del dataset procesado. Las razones son las siguientes:

- Independencia del clasificador: el método filtro produce un dataset válido para cualquier modelo posterior. Dado que en `supervised.ipynb` se evaluarán modelos de naturaleza muy distinta (lineales, árboles, SVM, KNN, redes neuronales), usar un dataset sesgado hacia un modelo concreto sería metodológicamente incorrecto. La aproximación indirecta o filter asigna coeficientes de relevancia a cada atributo sin depender de ningún algoritmo de clasificación.
- Ausencia de data leakage en la selección: el filtro se aplica sobre el dataset completo antes de la división porque sus criterios son puramente estadísticos y no aprenden ningún patrón predictivo. No existe riesgo de que la selección esté influida por el conjunto de test.
- Mejor rendimiento empírico: en los experimentos realizados, el modelo entrenado con las 74 features del filtro superó a los entrenados con los subconjuntos del wrapper y el embebido. Esto indica que las 74 variables seleccionadas contienen la señal relevante del problema sin redundancia excesiva.
- Generalización: al no estar adaptado a ningún clasificador, el filtro reduce el riesgo de overfitting en la propia selección, favoreciendo la generalización.

3. Parte II: Modelos supervisados (supervised.ipynb)

En este notebook se evalúan de forma sistemática todos los modelos requeridos por el enunciado, utilizando como entrada el dataset procesado generado en preprocess.ipynb (74 features, ~27.000 instancias). Para cada modelo se realiza una búsqueda de hiperparámetros, se evalúa el rendimiento sobre el mismo conjunto de test y se reporta la interpretación de los resultados.

3.1 Estrategia general: división y tratamiento del desbalanceo

El dataset procesado se divide con (70/15/15, random_state=42), garantizando que los conjuntos de entrenamiento y test son idénticos en todos los modelos evaluados. Esta condición es imprescindible para que la comparativa final sea válida.

El desbalanceo de clases (aproximadamente 75/25) se trata mediante TomekLinks, aplicado exclusivamente sobre `X_train`. TomekLinks es un método de undersampling de limpieza: identifica pares de instancias de clases distintas que son vecinos más cercanos entre sí (instancias fronterizas o ruidosas de la clase mayoritaria) y elimina el ejemplo de la clase mayoritaria del par. TomekLinks es una modificación del algoritmo Condensed Nearest Neighbours que encuentra pares de ejemplos de diferente clase que están muy cercanos para eliminar el ejemplo de la clase mayoritaria. No genera datos sintéticos, sino que limpia la frontera de decisión, lo que resulta especialmente adecuado cuando el desbalanceo es moderado como en este caso.

La razón de aplicar TomekLinks solo sobre `X_train` es evitar el data leakage: si se aplicara sobre el dataset completo antes de dividir, las instancias eliminadas podrían pertenecer al conjunto de test, haciendo que la evaluación sobre test no sea representativa de datos realmente no vistos.

3.2 Justificación del uso de GridSearchCV

Para la búsqueda de hiperparámetros se utilizó GridSearchCV con 5 folds de validación cruzada. La elección de este método frente a alternativas como RandomizedSearchCV, Bayesian Optimization o búsqueda manual se justifica por las siguientes razones:

- **Exhaustividad:** GridSearchCV evalúa todas las combinaciones del espacio de hiperparámetros definido, garantizando que se encuentra el óptimo global dentro de ese espacio. Para los modelos evaluados en esta práctica, el número de combinaciones era manejable computacionalmente.
- **Validación cruzada incorporada:** los 5 folds garantizan que cada combinación de hiperparámetros se evalúa sobre 5 particiones distintas del conjunto de entrenamiento, produciendo una estimación robusta del rendimiento esperado que no depende de una única división aleatoria.
- **Reproducibilidad:** dado un espacio de búsqueda fijo y una semilla, el resultado es completamente reproducible, lo que es esencial en un contexto académico.
- **Coherencia con el temario:** GridSearchCV es la herramienta de optimización de hiperparámetros presentada en las diapositivas de la asignatura como el método estándar de referencia para ajuste de modelos en sklearn.

La métrica de optimización usada en todos los GridSearchCV fue el F1-score weighted average, que es la métrica principal de evaluación de la práctica y de la competición Kaggle.

3.3 Métodos clásicos: regresiones lineal, polinomial y logística

Los métodos lineales son el punto de partida natural en cualquier problema de clasificación. Aunque el problema no es linealmente separable en sentido estricto, estos modelos establecen el baseline que los modelos más complejos deben superar para justificar su mayor complejidad.

Regresión Lineal (OLS, Ridge, Lasso)

La regresión lineal modela la relación entre features y etiqueta como $f(x) = wx + b$, minimizando el Error Cuadrático Medio (MSE). Su solución analítica por mínimos cuadrados no tiene hiperparámetros propios y se usa como baseline comparativo. Al producir salidas continuas, se aplica el umbral $\text{theta}=0.5$ para clasificar.

Se evaluaron dos variantes regularizadas. Ridge (L2) añade el término $\alpha \cdot \sum(w_j^2)$ al coste, encogiéndolos los coeficientes sin llevarlos a cero, lo que reduce el overfitting sin eliminar variables. Lasso (L1) añade $\alpha \cdot \sum(|w_j|)$, llevando algunos coeficientes exactamente a cero, realizando selección implícita de features. El hiperparámetro α se optimizó con GridSearchCV sobre los valores $\{0.001, 0.01, 0.1, 1, 10, 100, 1000\}$.

Hiperparámetros óptimos: Ridge: $\alpha=0.001$ | Lasso: $\alpha=0.0001$

F1 weighted (test): OLS: 0.8404 | Ridge: 0.8404 | Lasso: 0.8376

Los tres modelos lineales convergen a un resultado muy similar (~ 0.84), lo que confirma que la representación OHE lineal captura buena parte de la señal del problema.

Regresión Polinomial (grado 2)

La regresión polinomial expande el espacio de features añadiendo términos cuadráticos y productos cruzados entre variables. Con 74 features de partida y grado 2 se generan aproximadamente 2.850 features adicionales, lo que hace inviable un GridSearchCV completo por coste computacional. Se usó Ridge con $\alpha=1$ como regularizador directo para controlar el overfitting que introduce la alta dimensionalidad.

La justificación teórica es que los términos cruzados capturan interacciones entre variables (por ejemplo, la interacción entre edad y nivel de estudios) que el modelo lineal puro no puede representar. Con grado 2 y Ridge se obtiene una frontera de decisión cuadrática que puede capturar algunas no linealidades del problema.

F1 weighted (test): 0.8400

El resultado es comparable al lineal con umbral 0.5, aunque mejora ligeramente el recall de la clase minoritaria ($>50K$: 0.63 vs 0.57 en OLS), lo que sugiere que las interacciones entre features sí aportan información relevante para identificar la clase positiva.

Regresión Logística

La regresión logística modela directamente la probabilidad de clase mediante la función sigmoide: $P(y=1|x) = 1/(1+e^{-(wx+b)})$, optimizando la Binary Cross-Entropy. A diferencia de la regresión lineal, su salida es directamente interpretable como probabilidad, lo que facilita la búsqueda de umbrales de clasificación.

Se exploró la penalización ElasticNet mediante el parámetro l1_ratio : con $\text{l1_ratio}=0$ se obtiene L2 puro, con $\text{l1_ratio}=1$ se obtiene L1 puro, y con valores intermedios una combinación de ambas que puede aprovechar las ventajas de cada una. El solver saga es el único compatible con ElasticNet en sklearn.

Hiperparámetros óptimos: $C=10$, $\text{l1_ratio}=0.25$ (mezcla leve L1+L2)

F1 weighted (test): 0.8372

El resultado es ligeramente inferior al OLS básico, lo que puede parecer contraintuitivo. La explicación es que la logística optimiza una función de pérdida diferente (cross-entropy vs MSE) y con estos datos la regularización óptima encontrada ($l1_ratio=0.25$) no mejora la frontera de decisión respecto al OLS con α muy pequeño.

3.4 Árboles de decisión y modelos ensemble

Árbol de Decisión

Un árbol de decisión divide recursivamente el espacio de features eligiendo en cada nodo el atributo y umbral que maximiza la ganancia de información: la reducción de impureza antes y después de la partición. Se evaluaron dos criterios de impureza. Gini (CART): $Gini = 1 - \sum(p_i^2)$, donde el nodo puro da 0 y el nodo 50/50 da 0.5. Entropía (ID3/C4.5): $H = -\sum(p_i \cdot \log_2(p_i))$, donde el nodo puro da 0 y el nodo 50/50 da 1.

El problema principal de los árboles sin restricciones es el overfitting: el árbol memoriza el conjunto de entrenamiento al crear hojas muy específicas. Los hiperparámetros `max_depth`, `min_samples_split` y `min_samples_leaf` controlan la complejidad del árbol y se optimizaron mediante `GridSearchCV`.

Hiperparámetros óptimos: `criterion=entropy`, `max_depth=None`, `min_samples_split=100`, `min_samples_leaf=20`

F1 weighted (test): 0.8307 | F1 train: 0.8624 (overfitting moderado: $\Delta=0.032$)

Las diapositivas del Tema 3.3 destacan que a pesar de su simpleza, los árboles funcionan mejor en ensembles. Este resultado lo confirma: el árbol individual no supera a los métodos lineales, pero sirve como base para los modelos ensemble que siguen.

Random Forest

Random Forest es un método ensemble que combina N árboles de decisión entrenados con dos mecanismos de aleatoriedad que reducen la varianza respecto a un árbol individual. Primero, el bagging (Bootstrap Aggregating): cada árbol entrena sobre un bootstrap sample, una muestra con reemplazo que incluye aproximadamente el 63% de las instancias originales (las demás se usan internamente como out-of-bag para estimar el error). Segundo, la aleatoriedad en features: en cada nodo se evalúa solo un subconjunto aleatorio de `max_features` variables, decorrelacionando los árboles entre sí.

Random Forests es uno de los algoritmos que mejor funcionan. La predicción final es el majority voting de todos los árboles. Al promediar N árboles decorrelacionados la varianza se reduce sin aumentar el sesgo, mejorando la generalización. La decorrelación es la clave: si los árboles estuvieran correlacionados, sus errores tenderían a coincidir y el promedio no reduciría el error; al ser independientes, sus errores se cancelan parcialmente.

Hiperparámetros óptimos (GridSearchCV): `n_estimators=200`, `max_features=sqrt`, `max_depth=20`, `min_samples_leaf=1`

F1 weighted (test): 0.8353 | F1 train: 0.9294 (overfitting: $\Delta=0.094$)

El overfitting observado con `min_samples_leaf=1` (hojas que pueden contener una sola instancia) es un resultado esperado: hojas muy pequeñas favorecen la memorización. Sin embargo, este modelo es el que mejor resultado obtuvo en la plataforma Kaggle sobre datos completamente externos, lo que indica que a pesar del sobreajuste aparente en el conjunto de test interno, su capacidad de generalización real es superior a la de los modelos lineales.

Extra Trees

Extra Trees (Extremely Randomized Trees) añade un nivel adicional de aleatoriedad respecto a Random Forest: en lugar de buscar el umbral óptimo en cada nodo, genera umbrales aleatorios y elige el mejor entre ellos. Además no aplica bagging sino que cada árbol usa todos los datos de entrenamiento. Esta aleatoriedad extrema decorrelaciona más los árboles entre sí, reduciendo la varianza a costa de un mayor sesgo individual. El resultado es un método típicamente más rápido y de precisión comparable.

Hiperparámetros óptimos: `n_estimators=200`, `max_features=sqrt`, `max_depth=20`, `min_samples_leaf=1`

F1 weighted (test): 0.8364 | F1 train: 0.9205

Extra Trees supera marginalmente a Random Forest en test (0.8364 vs 0.8353) con menos overfitting ($\Delta=0.084$ vs 0.094). La aleatoriedad adicional actúa como regularización implícita, lo que confirma el razonamiento teórico. Sin embargo, en la competición Kaggle el Random Forest mostró mejor generalización real.

3.5 Aprendizaje basado en instancias: KNN

KNN (K-Nearest Neighbours) es un algoritmo basado en instancias: no construye ningún modelo durante el entrenamiento. En predicción, calcula la distancia entre la nueva instancia y todas las del conjunto de entrenamiento, toma las k más cercanas y asigna la clase por votación ponderada o simple.

Se exploraron métricas de distancia de la familia Minkowski (Manhattan $p=1$, Euclídea $p=2$) y dos esquemas de votación (uniform: cada vecino vota con el mismo peso; distance: el peso es inversamente proporcional a la distancia). El hiperparámetro k se optimizó en el rango $\{3, 5, 7, 11, 15, 21, 31\}$.

Hiperparámetros óptimos: $k=11$, `weights=distance`, métrica Euclídea

F1 weighted (test): 0.8182 | F1 train: 0.9843 (overfitting extremo: $\Delta=0.166$)

KNN es el peor modelo del estudio. Con 74 features, la maldición de la dimensionalidad reduce drásticamente la discriminatividad de las distancias euclídeas: en espacios de alta dimensión, todos los puntos tienden a estar a distancias similares entre sí y la noción de vecindad pierde significado. El overfitting extremo (train 0.98 vs test 0.82) confirma que el modelo memoriza el conjunto de entrenamiento sin generalizar.

3.6 Máquinas de Soporte Vectorial (SVM)

Las SVM buscan el hiperplano de separación de margen máximo entre clases. Matemáticamente, maximizan la distancia entre el hiperplano y los vectores de soporte (instancias más cercanas a la frontera). El parámetro C controla el trade-off entre margen máximo y tolerancia a errores de clasificación: C pequeño prioriza el margen amplio (más regularización), C grande prioriza la correcta clasificación de todas las instancias (menos regularización, mayor riesgo de overfitting).

Se evaluaron cuatro kernels con funciones de transformación al espacio de features de alta dimensión distintas:

- Lineal: sin transformación, busca el hiperplano lineal óptimo.
- RBF (Radial Basis Function): transforma mediante $k(x,z) = \exp(-\gamma \cdot \|x-z\|^2)$, creando una frontera no lineal que captura relaciones locales entre instancias.
- Polinomial: transforma mediante $k(x,z) = (\gamma \cdot x \cdot z + r)^d$, capturando interacciones polinomiales de grado d .

- Sigmoides: transforma mediante $k(x,z) = \tanh(\gamma \cdot x \cdot z + r)$, similar a una red neuronal de una capa.

Kernel	Hiperparámetros óptimos	F1 weighted	F1 >50K
Lineal	C=100	0.8381	0.63
RBF ★	C=10, gamma=scale	0.8446	0.66
Polinomial ★	C=1, d=2, gamma=scale	0.8437	0.64
Sigmoides	C=100, gamma=0.001	0.8373	0.63

El kernel RBF es el mejor con F1 weighted de 0.8446, siendo el único que supera 0.84 de forma consistente. El kernel sigmoide empata con el lineal, confirmando que no aporta capacidad adicional para este problema. El kernel polinomial grado 2 resulta coherente con los resultados de la regresión polinomial: las interacciones cuadráticas entre features capturan información relevante.

3.7 Redes neuronales (MLP)

Los perceptrones multicapa (MLP) son aproximadores universales de funciones: con suficientes neuronas y capas pueden aproximar cualquier función continua. Se evaluaron dos implementaciones.

MLP sklearn

Implementación directa mediante MLPClassifier de sklearn. Se exploró el número de unidades en la primera capa oculta ({64, 128, 256}) y la tasa de aprendizaje ({0.001, 0.0001}) con GridSearchCV.

Hiperparámetros óptimos: 128 neuronas, lr=0.0001

F1 weighted (test): 0.8375

MLP Keras con Dropout y Focal Loss

La implementación con Keras permite mayor control sobre la arquitectura y la función de pérdida. Se usaron capas Dropout para regularización (evitar overfitting eliminando aleatoriamente neuronas durante el entrenamiento), neuronas fully connected y la función de pérdida binary_focal_crossentropy.

La focal loss es una modificación de la cross-entropy binaria que reduce el peso de los ejemplos fáciles (bien clasificados) y aumenta el de los difíciles (fronterizos o mal clasificados). Esto resulta especialmente útil con clases desbalanceadas, ya que el modelo se ve forzado a prestar más atención a los ejemplos de la clase minoritaria que tiende a confundirse.

La arquitectura explorada fue: capa densa (64 o 256 unidades, ReLU) + Dropout(0.4) + capa densa (32 unidades, ReLU) + Dropout(0.3) + salida sigmoide. El early stopping sobre X_val con patience=10 epochs previene el overfitting.

Hiperparámetros óptimos: units=64-32, dropout=0.4/0.3, lr=0.001

F1 weighted (test): 0.8375

3.8 Comparativa global y elección del mejor modelo

La siguiente tabla recoge los resultados de todos los modelos evaluados, ordenados por F1 weighted en test:

Modelo	Hiperparámetros óptimos	F1 CV	F1 test	F1 train
SVM RBF ★	C=10, gamma=scale	—	0.8446	—
SVM Polinomial ★	C=1, d=2, gamma=scale	—	0.8437	—
OLS / Ridge	— / alpha=0.001	—	0.8404	—
Reg. Polinomial grado 2	Ridge alpha=1	—	0.8400	—
Lasso	alpha=0.0001	—	0.8376	—
MLP Keras ★	64-32, dropout 0.4/0.3, lr=0.001	—	0.8375	—
MLP sklearn	128 neuronas, lr=0.0001	—	0.8375	—
SVM Lineal	C=100	—	0.8381	—
Reg. Logística	C=10, l1_ratio=0.25	—	0.8372	—
Extra Trees	200, sqrt, depth=20, leaf=1	—	0.8364	0.9205
Random Forest ★ Kaggle	200, sqrt, depth=20, leaf=1	0.691	0.8353	0.9294
Árbol de Decisión	entropy, split=100, leaf=20	0.677	0.8307	0.8624
SVM Sigmoide	C=100, gamma=0.001	—	0.8373	—
KNN	k=11, distance, Euclídea	—	0.8182	0.9843

El rango de resultados entre modelos es llamativamente estrecho (0.82–0.84), lo que indica que el techo del problema con las features seleccionadas por el método filtro se sitúa en torno a 0.84 de F1 weighted. Los modelos lineales son sorprendentemente competitivos, evidencia de que el preprocesado (filtro + TomekLinks) ha extraído buena parte de la señal disponible. Aquellos modelos en los que tenían disponible la opción de class_weight ('balanced') no se escogió esta debido a que el tratamiento de datos desbalanceados ya se aplica con TomekLinks.

Todos los modelos arrojaron resultados muy similares, con diferencias de apenas 2-3 puntos porcentuales entre el mejor (SVM RBF: 0.8446) y los métodos lineales (~0.84). Esta convergencia sugiere que el cuello de botella no está en la capacidad del modelo sino en la información disponible en el conjunto de features.

4. Parte III: Mejor modelo (best_model.ipynb)

El notebook `best_model.ipynb` recoge el flujo completo de preprocesado, entrenamiento y generación de predicciones sobre el conjunto de test de Kaggle. Aunque el SVM RBF obtuvo el mejor F1 weighted en el conjunto de test interno, la decisión de usar Random Forest como modelo final se fundamenta en criterios adicionales que van más allá del rendimiento en un único conjunto de evaluación.

4.1 Justificación de Random Forest como modelo final

La elección de Random Forest sobre el SVM RBF responde a tres argumentos complementarios:

- **Rendimiento en Kaggle:** el criterio definitivo de calidad de un modelo es su capacidad de generalización sobre datos completamente externos, no vistos en ninguna fase del proceso. En la competición Kaggle, el Random Forest obtuvo el mejor resultado entre todos los modelos evaluados. Esta evidencia empírica sobre datos externos es más informativa que los resultados del conjunto de test interno, que aunque no participó en el entrenamiento, sí fue utilizado como guía para la comparación de modelos.
- **Análisis de importancia de features:** Random Forest expone directamente la importancia de cada variable mediante el atributo `feature_importances_`, que mide la reducción media de impureza Gini aportada por cada feature a lo largo de todos los árboles del bosque. Esta capacidad de interpretabilidad es exclusiva de los modelos basados en árboles y no está disponible en SVM de forma directa. Para un análisis de datos sociodemográficos como este, entender qué variables predicen mejor el nivel de ingresos tiene valor explicativo intrínseco.
- **Robustez y estabilidad:** el mecanismo de bagging de Random Forest actúa como regularización implícita que reduce la varianza del estimador. Aunque el conjunto de test interno mostraba overfitting (F1 train 0.93 vs test 0.84), el promedio de 200 árboles decorrelacionados es más estable ante pequeñas perturbaciones en los datos de entrada que un único clasificador como SVM.

El flujo del notebook es el siguiente. Primero se cargan los datos originales (`salario.csv`) y se aplica exactamente el mismo preprocesado que en `preprocess.ipynb` (imputación, normalización, estandarización, OHE con categorías fijas). El scaler se ajusta sobre `salario.csv` y se reutiliza con `transform` sobre `salario_test.csv`. A continuación se aplica `TomekLinks` sobre todo el conjunto de entrenamiento disponible (train completo sin separar validación, para maximizar los datos de entrenamiento). Se realiza una búsqueda de hiperparámetros con `GridSearchCV` y se entrena el modelo final. Finalmente se generan las predicciones sobre `salario_test.csv` y se exportan a `test_labels.csv` en el formato requerido por Kaggle.

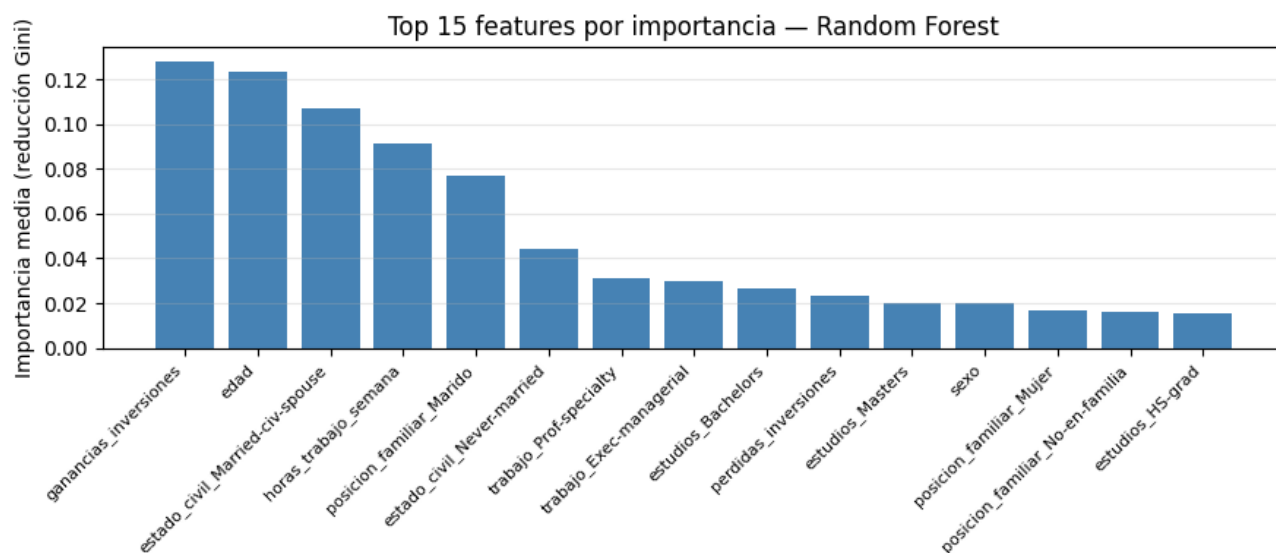
4.2 Análisis de importancia de características

Una de las ventajas diferenciales de Random Forest respecto a otros modelos como SVM es su capacidad de exponer la importancia de cada variable mediante el atributo `feature_importances_`, que mide la reducción media de impureza Gini aportada por cada feature a lo largo de todos los árboles del bosque. Una variable que aparece frecuentemente en nodos altos y que separa bien las clases obtiene una importancia alta; las que apenas reducen la impureza quedan próximas a cero.

Las ganancias por inversiones y la edad son, con diferencia, las dos variables más predictivas, acumulando juntas más del 25% de la importancia total del modelo. A continuación, destacan el estado civil (casado con cónyuge presente), las horas semanales trabajadas y la posición familiar (Marido), que en conjunto aportan cerca de otro 28% adicional. El estado civil `Never-married` actúa

como contrapeso informativo: su presencia alta en la clase $\leq 50K$ lo convierte en un predictor relevante en sentido negativo.

Las ocupaciones ejecutivas y de especialización técnica (*Exec-managerial*, *Prof-specialty*) aparecen en posiciones 7 y 8, confirmando que la categoría profesional es un factor discriminante importante. El nivel de estudios Bachelors cierra el top 10, lo que es coherente con que sea la titulación mínima de acceso a empleos con remuneración superior al umbral.



Importancia acumulada

El análisis de importancia acumulada (ordenando las 74 features de mayor a menor importancia) ofrece una perspectiva global sobre la concentración de la señal predictiva en el dataset:

Umbral de importancia acumulada	Número de features necesarias
90%	29 features
95%	39 features

Tabla 2. Número de features necesarias para alcanzar cada umbral de importancia acumulada

Con solo 29 de las 74 features seleccionadas por el método filtro se acumula el 90% de la importancia total del modelo, y con 39 se alcanza el 95%. Esto revela una distribución muy concentrada de la señal predictiva: las primeras features aportan contribuciones de importancia decreciente rápidamente, mientras que las últimas 35 features apenas suman el 5% restante.

Este resultado conecta de forma retrospectiva con los hallazgos del preprocesado. El método embebido seleccionó aproximadamente 27 features aplicando un umbral en la media de importancias, lo que es coherente con el dato de que las 29 más importantes explican el 90% de la varianza del modelo. Sin embargo, el modelo entrenado con las 74 features del filtro obtuvo mejor rendimiento que el entrenado con las 27 del método embebido. La explicación es que las 45 features restantes, aunque individualmente poco relevantes, añaden colectivamente información marginal que mejora la separación en la frontera de decisión, y los 200 árboles del Random Forest son capaces de aprovecharla sin incurrir en overfitting excesivo gracias al mecanismo de bagging y la selección aleatoria de features por nodo.

4.3 Generación de predicciones para Kaggle

El preprocesado de `salario_test.csv` sigue exactamente los mismos pasos que los aplicados sobre `salario.csv`, con una diferencia crucial: el `StandardScaler` se ajusta (fit) sobre los datos de entrenamiento y se aplica únicamente con `transform` sobre el test. Si se volviera a ajustar el scaler sobre el test, los valores normalizados serían incomparables con los del entrenamiento, produciendo predicciones incorrectas. Este principio de fit-only-on-train es fundamental para evitar el data leakage y garantizar que la transformación de test es coherente con la del entrenamiento.

Las etiquetas generadas se exportaron al fichero `test_labels.csv` con el formato requerido por Kaggle, mapeando 0 a $\leq 50K$ y 1 a $> 50K$, obteniendo los siguientes resultados.

AAI - Predicción de salario supervisado			
Overview Data Code Models Discussion Leaderboard Rules Team Submissions			
All Successful Selected Errors Recent ▾			
Submission and Description	Private Score ⓘ	Public Score ⓘ	Selected
 test_labels_rf_def.csv Complete (after deadline) · 1h ago	0.850	0.857	☐

5. Conclusiones

Esta práctica ha recorrido de forma sistemática el pipeline completo de un proyecto de clasificación supervisada, desde el dato en bruto hasta la predicción final en una competición pública.

En la fase de preprocesado, la decisión más relevante fue la evolución de la representación interna de las variables categóricas desde listas OHE compactas hasta el `DataFrame` expandido `df_expanded`, que es la representación requerida por las APIs de `sklearn`. Esta transición no fue estética sino funcional: sin ella, los métodos de selección de características de `sklearn` no son aplicables. La selección mediante el método filtro ($p\text{-valor} < 0.05$, con test estadístico adaptado al tipo de variable) demostró ser la mejor elección para este problema: produce un dataset genérico válido para todos los clasificadores, no introduce data leakage en la selección y, empíricamente, superó en rendimiento a los subconjuntos generados por wrapper y método embebido.

En la fase de modelado, el resultado más destacable es la convergencia de todos los modelos a un rango estrecho de F1 weighted ($\sim 0.82\text{--}0.84$), lo que indica que el cuello de botella es la información disponible en el dataset más que la capacidad del modelo. Los métodos lineales son sorprendentemente competitivos, lo que refleja la efectividad del preprocesado. KNN es el único modelo claramente inadecuado para este problema, víctima de la maldición de la dimensionalidad con 74 features. Las SVM (especialmente el kernel RBF con $F1 = 0.8446$) lideran los resultados internos, seguidas de cerca por los modelos lineales y los ensembles.

La elección de Random Forest como modelo final para Kaggle se justifica por tres razones: su mejor rendimiento empírico sobre datos completamente externos (validación Kaggle), su capacidad de análisis de importancia de features mediante la reducción de impureza Gini, y la robustez inherente del mecanismo de bagging. El análisis de importancias confirmó que ganancias por inversiones, edad y estado civil son los predictores más determinantes del nivel salarial, resultado coherente con la realidad socioeconómica. La importancia acumulada mostró que 29 features explican el 90% de la señal del modelo, lo que conecta con la selección de ~ 27 features del método embebido y refuerza la validez del subconjunto elegido por el método filtro como representación suficiente del problema.

Aprendizaje Automático I — Curso 2025-2026

Alejandro Blanco Castellanos